

Modus Primus

Engineering Specification for AI Architecture

(PAHA Companion)

Version 1.1

James (JD) Longmire
Northrop Grumman Fellow (unaffiliated research)
Chief Architect, Digital Ecosystems
ORCID: 0009-0009-1383-7698
Correspondence: jdlongmire@outlook.com

February 2026

Cited Architectural Framework
Longmire, J. (2026). *Portable Agent Harness Architecture* (Rev 2.2)
[doi:10.5281/zenodo.20112631](https://doi.org/10.5281/zenodo.20112631)

Front Matter Note: General Guidance for AI Engineering Practice

This specification establishes a documented engineering practice for producing and maintaining PAHA-conformant AI architectures. The guidance below describes the document's intended use; it is not itself part of the specification.

This document is structured for two reader contexts. The first is an enterprise with an existing systems engineering management plan (eSEMP) or equivalent engineering practice; in this context, the document is integrated into the enterprise's existing practice, with general engineering processes (configuration management, organizational baseline, technical reviews, risk management, V&V methodology) inherited from the enterprise's existing framework and the AI-specific specializations specified here layered on top. The second is an enterprise without an existing eSEMP; in this context, the document serves as the engineering specification directly, and the enterprise treats the AI-specific specializations as part of a broader engineering practice it will need to maintain.

The conventions adopted in this document are: light process detail with specialization where the AI architecture domain requires it; hierarchical section numbering aligned with INCOSE engineering practice; appendices for detailed technical baseline and supporting specifications; Harvard-style citations in references; explicit [ENTERPRISE:] placeholders where enterprise-specific decisions must be made (effective date, document identifier convention, organizational roles, review cadences). The placeholders are grep-able and easy to find during editorial completion.

The document is not vendor-specific. It does not assume any particular cloud provider, foundation model, orchestration platform, or governance toolchain. The PAHA architectural commitments specify what the architecture must achieve; this specification specifies the engineering practice that produces it; the enterprise's tech baseline specifies which specific products and services fulfill the architecture in its operational context.

1. Introduction and Scope

1.1 AI Architecture as a Systems Engineering Domain

AI architecture is a technical domain that introduces engineering concerns most general systems engineering practices do not fully address: substrate volatility (foundation models and inference infrastructure evolve on cycles measured in months, not years), behavioral non-determinism (probabilistic substrates produce outputs within bounded variation rather than deterministic results), federated governance across security enclaves (deployment patterns must accommodate hard security boundaries while preserving architectural coherence), and runtime assurance properties (continuous monitoring of

drift, mission coherence, policy deviation, and explainability as architectural commitments rather than incidental monitoring). These concerns require engineering specialization beyond what general systems engineering practice provides.

This specification establishes the engineering practice for producing and maintaining PAHA-conformant AI architectures. It is the engineering companion to the Portable Agent Harness Architecture pattern (PAHA), which specifies the architectural pattern that governance, sovereignty, and multi-enclave constraints require. PAHA establishes what the architecture must achieve; this specification establishes how the enterprise's engineering practice produces it.

The specification's structure is consistent with established systems engineering and architecture-description conventions. The mapping below identifies the principal correspondences; the specification does not claim to be a strict implementation of any single framework, and where this specification's terminology differs from a framework's, the specification's terminology is authoritative within its own scope.

Modus Primus concept	External standard analogue	Notes
WBS (Section 3, Appendix B)	INCOSE system breakdown; ISO/IEC/IEEE 42010 architecture description	The WBS is the architectural specification; per-enclave tech baselines are realizations of it
Five-M decomposition (Section 2.3)	ISO/IEC/IEEE 42010 architecture viewpoints	Each M is a viewpoint on the system's purposive disposition
Federation pattern (Section 4)	TOGAF federation; ISO/IEC/IEEE 42010 architecture frameworks	Tier-bounded inheritance contracts with delegation and escalation paths
V&V instruments (Section 7)	INCOSE V&V; ISO/IEC/IEEE 15288 verification and validation	Specialized for substrate volatility, non-determinism, and continuous mission alignment
Runtime assurance (Section 7.4)	Runtime verification; continuous IV&V	AI-architecture-specific drift, coherence, deviation, and explainability monitoring
Trust escalation model	Authority-to-Operate progression; risk-based access	Governance-plane primitive tying execution-policy to execution-runtime

1.2 Integration with Enterprise Engineering Practice

This specification is the companion to PAHA, not a subordinate document. It does not assume the enterprise has a particular systems engineering management plan, configuration management

framework, or organizational baseline. It assumes only that the enterprise has decided to implement PAHA-conformant AI architecture and needs an engineering specification to do so.

Two integration contexts are anticipated, and the specification serves both.

The first context is an enterprise with an existing systems engineering management plan (eSEMP) or equivalent engineering practice. In this context, this specification integrates into the enterprise's existing practice. General engineering processes (configuration management, organizational baseline, broad technical reviews, risk management, V&V methodology) are inherited from the enterprise's existing framework; the AI-specific specializations specified here are layered on top. The enterprise's existing process owners apply their own organizational conventions to the AI architecture domain, using this specification as the source of AI-specific requirements and constraints.

The second context is an enterprise without an existing eSEMP. In this context, the specification serves as the engineering specification directly. The enterprise treats the AI-specific specializations as part of a broader engineering practice it will need to maintain. Sections that this specification declares as inheritance-heavy (general configuration management, broad technical reviews, organizational baseline) must be supplied by enterprise practice as the specification matures into use; the AI-specific specializations are not affected by whether or not this scaffolding exists.

The specification is structured to make both contexts viable without forcing one or the other. Where general engineering practice is referenced, it is referenced abstractly rather than to a specific parent document. Enterprise-specific decisions are marked with [ENTERPRISE:] placeholders that flag where the enterprise must supply its own conventions.

1.3 Relationship to PAHA

This specification cites Portable Agent Harness Architecture (Longmire 2026; doi:10.5281/zenodo.20112631) as the architectural framework it implements. PAHA establishes the architectural pattern for sovereignty-bounded enterprise AI deployments; this specification establishes the engineering practice that produces and maintains PAHA-conformant AI architectures.

PAHA is referenced rather than reproduced. Appendix A summarizes the PAHA elements relevant to this specification; readers needing the full PAHA specification should consult the PAHA paper directly. The relationship between PAHA and this specification is implementation-companion: PAHA is the architectural pattern, this specification is the engineering practice that operationalizes the pattern.

The companion relationship has practical consequences. Changes to PAHA may require revisions to this specification; changes to this specification do not require revisions to PAHA. Where the two documents appear to be in tension, PAHA's architectural commitments take precedence; this specification adapts. Where this specification appears to require more than PAHA, the additional content is engineering practice that PAHA does not specify, not an extension of the architectural pattern.

The principal correspondences between PAHA primitives and Modus Primus artifacts are summarized below; Appendix A provides the complete mapping.

PAHA primitive	Modus Primus artifact	Section
Meta-harness (durable architectural layer)	Five-M decomposition in mind.md, morals.md, mission.md, memory.md, means.md	Section 2.3, Appendix B.3
Three-plane decomposition (governance, cognitive, operational)	WBS layer architecture	Section 3, Appendix B
Seven minimum viable harness services	Mechanism Layer plus execution-runtime	Sections 3.6, 3.7.2, Appendix B.7, B.8.2
Enclave constraint and federation pattern	Five Modus tiers with inheritance, delegation, and escalation	Section 4, Appendix B.9
Capability registry (purposive election)	means/mechanisms selection relation	Section 2, Appendix C
Trust escalation as governance primitive	Trust escalation model in execution-policy	Sections 2.5, 3.7.1, Appendix B.8.1
Substrate adapter abstraction	Substrate substitution V&V	Section 7.5

1.4 What This Specification Provides

This specification provides seven distinct types of content.

The architectural principles that organize the AI architecture domain (Section 2). The means/mechanisms distinction is the foundational principle and propagates throughout the document.

The technical baseline specification (Section 3 with Appendix B). The Work Breakdown Structure specifies the artifacts that constitute a Modus Primus instance.

The federation pattern (Section 4). The Modus tier model specifies how AI architecture instances relate across security enclaves.

The technical baseline management approach (Section 5). The relationship between the architectural specification and per-enclave tech baselines is the configuration management specialization for AI architecture.

Engineering processes specific to AI architecture (Section 6). Substrate volatility, agent contract authoring, means election, federation schema evolution, and runtime assurance engineering are the processes this specification addresses.

V&V methodology (Section 7). Four V&V instruments specialize general V&V approaches for AI architecture's distinctive properties.

Domain-specific risks, technical reviews, and organizational responsibilities (Sections 8, 9, 10). These sections specify AI-architecture-specific specializations; general engineering practice elements are referenced abstractly and supplied by enterprise context.

1.5 Reading Guide

This specification serves three audiences. Engineering audiences (systems engineering function, chief engineer's office) should read Sections 1, 2, 6, 7, and 9 as the engineering-specialization core. Governance audiences (architecture governance, compliance, security) should read Sections 1, 2, 4, 8, and 10 as the governance-relevant core. Program audiences (program managers and architects building specific AI capabilities) should read Sections 3, 4, 5, and Appendix B as the implementation-relevant core. The full document is coherent end-to-end, but each audience can engage its relevant sections without reading the others.

1.6 First Actions by Audience

The reading guide identifies which sections each audience should engage. The list below identifies the first concrete actions each audience typically takes after reading. The actions are descriptive of common adoption patterns rather than prescriptive; specific sequencing depends on enterprise context.

For engineering audiences, typical first actions include instantiating the WBS as a Modus Primus instance for the target architectural domain, populating the per-enclave tech baseline template (Appendix D) for the first enclave, and standing up the four V&V instruments (Section 7) at minimum viable fidelity. Standing up runtime assurance signals (Section 7.4) as a continuous evidence base is typically the longest-lead engineering effort and is best initiated early.

For governance audiences, typical first actions include approving ownership assignments for the five-M files (Section 10), defining trust escalation thresholds and triggers in `execution-policy.md`, setting review cadences for the WBS and per-enclave tech baselines (Section 9), and establishing the means election review process (Section 9.7) as a working governance forum. Naming the WBS owner and `means.md` owner is the prerequisite that unblocks most other governance activities.

For program audiences, typical first actions include authoring initial agent contracts (Appendix E) for the program's key Tertius agents, identifying candidate mechanisms in the Mechanism Layer that would benefit from purposive election as means, and integrating runtime assurance signals into program-level operational dashboards. Programs typically also establish federation contract obligations with their parent Modus Secundus instance early to surface schema and audit-aggregation requirements before they constrain delivery timelines.

1.7 Out of Scope

This specification establishes engineering practice for AI architecture. The following are explicitly out of scope and remain the responsibility of enterprise practice or other documents.

Detailed Identity and Access Management implementation. The specification cites IAM constraints and trust escalation primitives but does not specify identity provider choice, credential issuance procedures, or session management mechanics. These are properties of the enterprise's IAM infrastructure.

Specific model selection and tuning recipes. The specification cites the Cognitive Engine layer (Section 3.3) and the substrate substitution V&V (Section 7.5) but does not specify which foundation models to select, fine-tuning procedures, or prompt engineering practice. Those decisions are captured in per-enclave tech baselines (Appendix D), not in the WBS.

Detailed program management processes. The specification cites delivery cadences and review gates but does not specify schedule, cost, or staffing methodology. Program management is the enterprise's domain.

Vendor-specific tooling guides. The specification is vendor-neutral. Implementations using specific cloud providers, orchestration platforms, model APIs, or governance toolchains are documented in per-enclave tech baselines or in vendor-specific addenda, not in the canonical specification.

Training curriculum. The specification identifies competency requirements implicitly through role definitions (Section 10) but does not specify training content, certification pathways, or skills frameworks. Workforce development is enterprise-specific.

Operational runbooks. The specification establishes the discipline that runbooks must align to the meta-harness and execution-policy.md but does not provide runbook content. Runbooks are properties of specific deployments, not of the architectural pattern.

2. Architectural Principles

2.1 Means and Mechanisms

A *means* is purposive. It is what the system uses *toward* its mission within its morals. Means carry teleological weight: their existence is justified by what they advance, not by what they can do in isolation.

A *mechanism* is operationally agnostic. It is the machinery by which something works, independent of why. Mechanisms exist as available capacity. They become meaningful when elected by some purposive layer; until then they are inventory.

Means and mechanisms describe the same underlying reality from two different stances. The same code execution environment may be inventoried as a mechanism (the Mechanism Layer) and elected as a means (means.md). The first description tells what is available; the second tells what is purposively in use.

This distinction is the foundational architectural principle of the AI architecture domain. It organizes the WBS, the federation pattern, the V&V approach, and the engineering specializations. The full statement of the principle appears in Appendix C.

2.2 The Selection Relation

For each purposive declaration in the meta-harness layer, there is a corresponding mechanism layer that catalogs operationally available capacity. The relationship between them is *selection*, not *abstraction*.

Purposive declaration	Selects from	Mechanism layer
mind.md	reasoning machinery	Cognitive Engine (Section 3.3)
morals.md + execution-policy.md	enforcement machinery	execution-runtime.md (Section 3.7.2)
mission.md	sequencing machinery	Orchestration (Section 3.4)
memory.md	retention machinery	Observability infrastructure (Section 3.9)
means.md	operational machinery	Mechanism Layer (Section 3.6)

The pattern is consistent across the architecture: a purposive layer elects from a mechanism layer. Mechanisms not elected by any purposive layer are not part of the system's disposition even if they are operationally available.

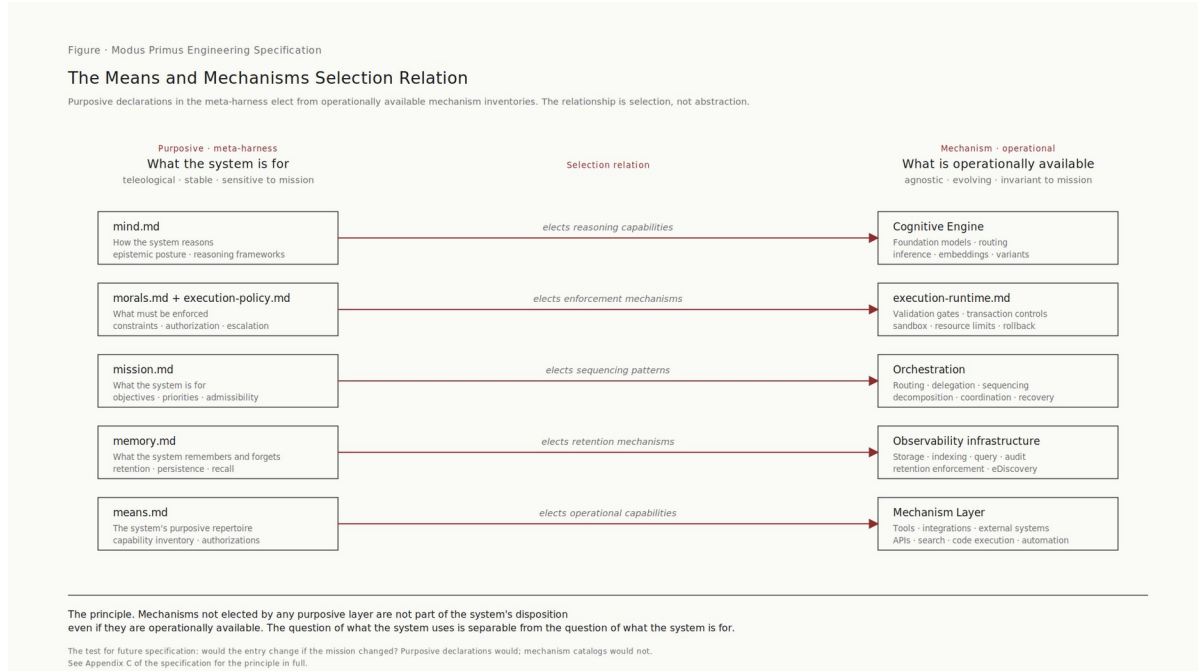


Figure 1. The Means and Mechanisms Selection Relation. Five purposive declarations in the meta-harness elect from operationally available mechanism inventories. The relationship is selection, not abstraction.

2.3 The Five-M Decomposition

The meta-harness layer comprises five files (mind, morals, mission, memory, means) that together declare the system's disposition toward its purpose. The order is hierarchical: mind grounds morals, morals grounds mission, mission grounds memory, memory grounds means. Reasoning posture grounds ethical constraint; ethical constraint grounds purpose; purpose determines what must be remembered; what is remembered grounds what the system uses to act. The five M's are unified by being purposive declarations rather than mechanism catalogs.

- **mind** declares how the system reasons toward its mission
- **morals** declares what the system must not do toward any end
- **mission** declares what the system is for
- **memory** declares what is worth remembering given what the system is for
- **means** declares the system's purposive repertoire for advancing the mission within the morals

If any of these slips into operational description, it stops being one of the M's. The five-M unity holds only when all five are read as purposive declarations.

2.4 Federation as Architectural Pattern

The architecture is federated rather than monolithic. Each security enclave hosts its own instance of the architecture, instantiated from a shared specification. Federation occurs at the schema and audit-aggregation layer through a shared federation contract; no operational data crosses enclave boundaries.

The federation pattern is hierarchical with safety bypasses. Delegation flows strictly through the tier hierarchy (Modus Primus to Secundus to Tertius to Quartus or Quintus). Escalation flows through the hierarchy by default, with defined conditions under which lower tiers may bypass intermediate tiers for safety-critical exceptions.

Section 4 specifies the federation pattern in operational detail.

2.5 Trust Escalation as a Governance Plane Primitive

Trust escalation, the progression from advisory recommendations to fully autonomous action, is a governance plane primitive enforced uniformly across all consoles, agents, and substrates within an enclave. The trust escalation model is not vendor-bound, substrate-bound, or console-bound. It is an architectural commitment that applies across the federation.

The trust escalation states are inherited from PAHA: advisory only, recommend-and-approve, automated-with-audit, fully autonomous. Each transition between states requires documented evidence and stakeholder approval. The transitions are governed by `execution-policy.md` and enforced by `execution-runtime.md`.

2.6 The Test for Future Specification

When adding to the architecture, ask: *is this a purposive declaration or a mechanism catalog?*

If purposive, it belongs in the meta-harness layer (or one of its declarative cousins like `execution-policy.md`). It should describe what the system uses *toward* its mission, not what is available.

If mechanism, it belongs in one of the mechanism layers (Cognitive Engine, Orchestration, Mechanism Layer, or `execution-runtime.md`). It should describe what is operationally available, not what is purposively elected.

When in doubt, the test is whether the entry would change if the mission changed. Purposive declarations would; mechanism catalogs would not.

3. Technical Baseline Specification

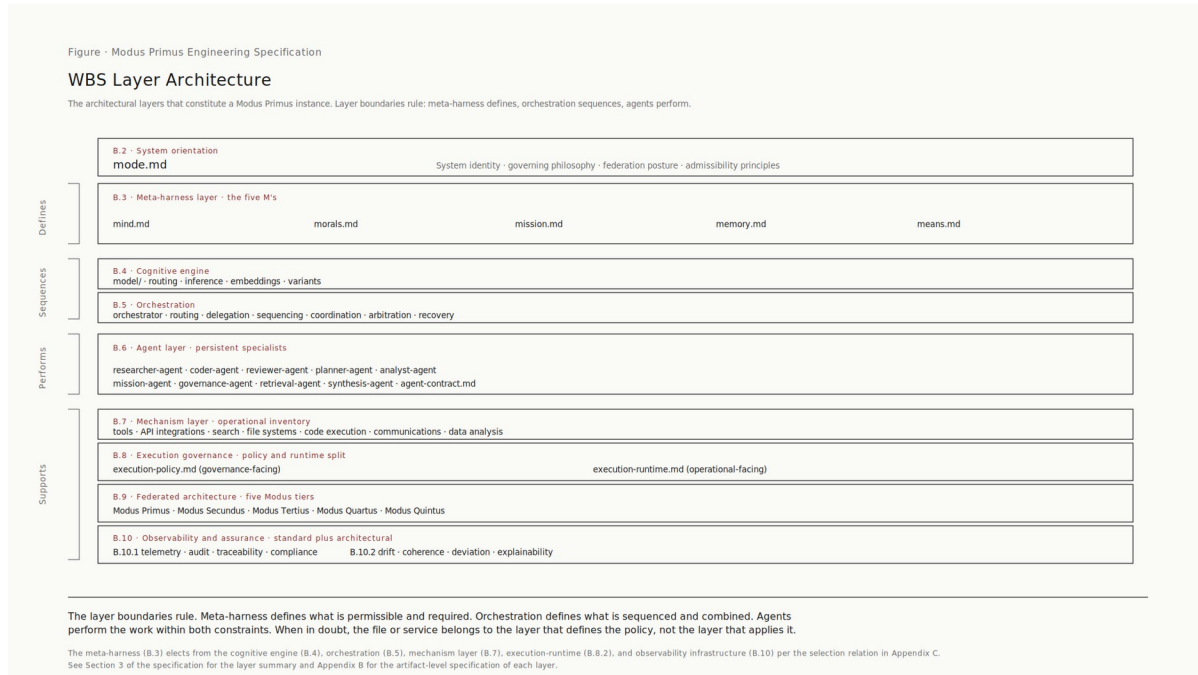


Figure 2. WBS Layer Architecture. The architectural layers that constitute a Modus Primus instance, organized by the layer boundaries rule: meta-harness defines, orchestration sequences, agents perform.

The technical baseline specification establishes the architectural artifacts that constitute a Modus Primus instance. The full WBS is in Appendix B; this section presents the layer summary.

3.1 System Orientation

A single artifact (mode.md) that establishes system identity, governing philosophy, and federation posture. Read at boot or session start; stable across runtime.

3.2 Meta-Harness Layer

The five-M decomposition: mind.md, morals.md, mission.md, memory.md, means.md. Each file declares one durable concern of the system's disposition. Files are declarative, not procedural.

3.3 Cognitive Engine Layer

The reasoning substrate over which agents operate. Catalogs foundation models, specialized models, multi-model routing, inference management, embedding systems, fine-tuned variants, and reasoning infrastructure. mind.md elects reasoning capabilities from this layer.

3.4 Orchestration Layer

The sequencing machinery. Catalogs orchestration patterns, routing logic, delegation mechanics, sequencing rules, coordination patterns, arbitration mechanics, load balancing, task decomposition, execution planning, and recovery handling. `mission.md` and the agent contracts elect orchestration patterns from this layer.

3.5 Agent Layer

The persistent specialist agents at the Modus Tertius tier. Catalog includes researcher-agent, coder-agent, reviewer-agent, planner-agent, analyst-agent, mission-agent, governance-agent, retrieval-agent, and synthesis-agent. Each agent is defined by an agent contract conforming to the format in Appendix E. Agent governance properties are inherited from the meta-harness; agent mechanics (lifecycle, identity, memory scope) are layer-specific.

3.6 Mechanism Layer

The tools, integrations, and external systems available to agents. Catalogs API integrations, search systems, file systems, code execution environments, communications channels, data analysis pipelines, scheduling systems, visualization systems, simulation systems, and automation pipelines. `means.md` elects operational capabilities from this layer.

3.7 Execution Governance Layer

Two artifacts: `execution-policy.md` (governance-facing, declarative) and `execution-runtime.md` (operational-facing, mechanical).

3.7.1 `execution-policy.md`

Declares what must be enforced. Validation rules, human approval gates, safety verification requirements, policy enforcement specification, risk assessment criteria, real-world action authorization, escalation triggers, approval workflow specifications, audit logging requirements, required compliance artifacts.

3.7.2 `execution-runtime.md`

Specifies how policy is enforced at runtime. Pre-action validation gates, approval workflow runtime, transaction controls, rollback procedures, environment isolation, sandbox enforcement, resource limit enforcement, real-world action gating, failure containment, runtime audit emission.

3.7.3 Audit Lifecycle

`morals.md` and `execution-policy.md` *declare* what must be audited; `execution-runtime.md` *emits* audit records; observability infrastructure (Section 3.9) *stores and queries* them; `memory.md` *declares retention* policy for the resulting records. The four touchpoints are ordered stages of a single lifecycle, not duplicate concerns.

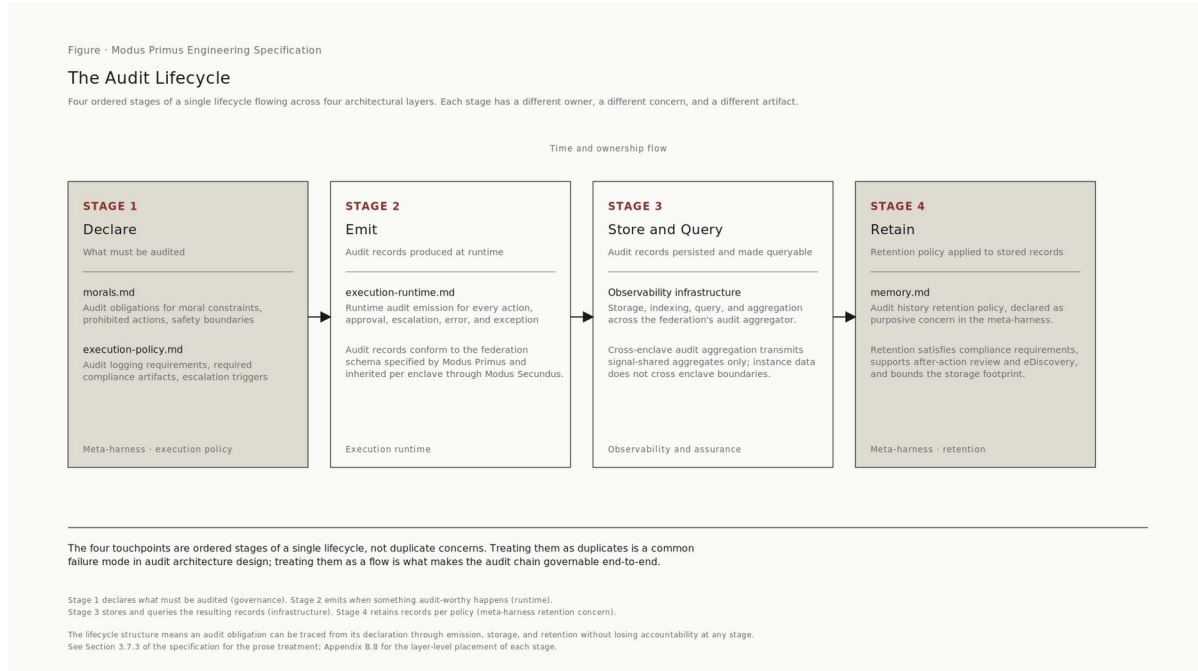


Figure 3. The Audit Lifecycle. Four ordered stages of a single lifecycle (declare, emit, store and query, retain) flowing across four architectural layers. Each stage has a different owner, concern, and artifact.

3.8 Federated Architecture Layer

The five Modus tiers (Primus, Secundus, Tertius, Quartus, Quintus). Specified in operational detail in Section 4.

3.9 Observability and Assurance Layer

Two subsections.

3.9.1 Standard Observability Infrastructure

Telemetry collection, auditability, traceability, compliance artifact collection, behavioral analytics. Necessary infrastructure inherited from typical enterprise observability practice.

3.9.2 Architectural Runtime Assurance

Drift detection, mission coherence monitoring, policy deviation detection, explainability surfacing. These are architectural commitments specific to AI architecture and serve as continuous V&V instruments (see Section 7).

4. Federation Specification

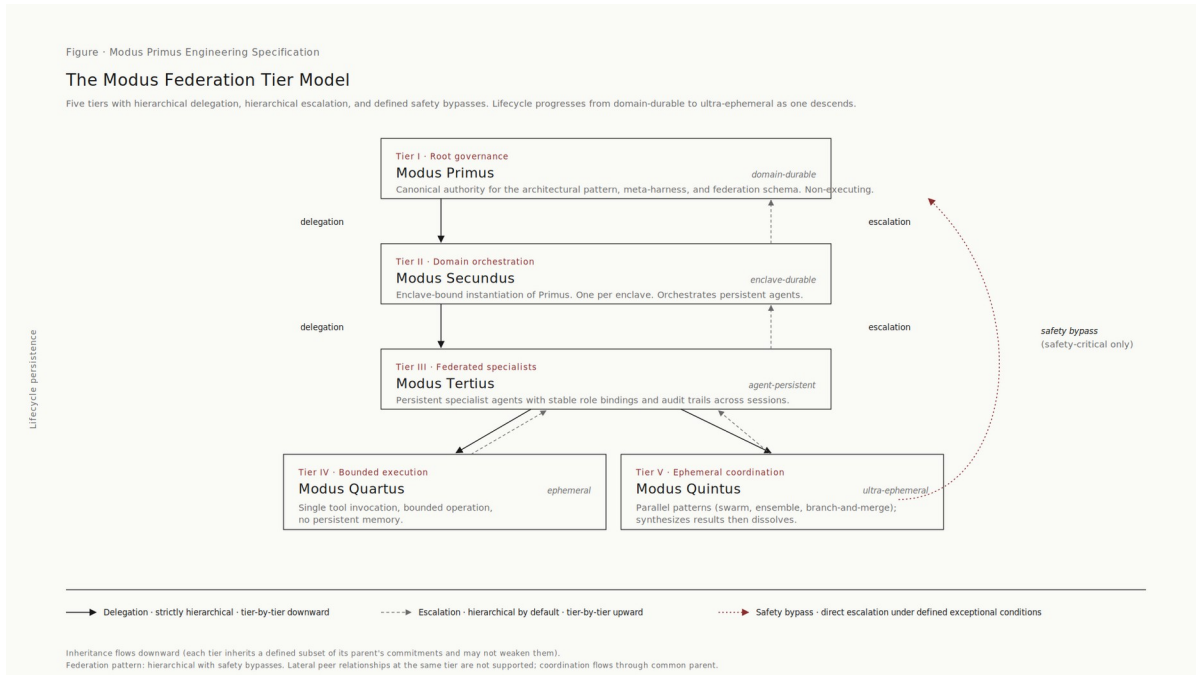


Figure 4. The Modus Federation Tier Model. Five tiers with hierarchical delegation, hierarchical escalation, and defined safety bypasses. Lifecycle persistence decreases as one descends from Modus Primus to Modus Quintus.

4.1 The Five Tiers

Modus Primus is the canonical authority for the architectural pattern, the meta-harness specification, and the federation schema. One Modus Primus per architectural domain. Does not execute work; defines the conditions under which work is admissible.

Modus Secundus is the enclave-bound instantiation of Modus Primus. Each enclave hosts one Secundus instance operating under Primus specification within the enclave's authorization scope. Secundus instances are operationally separate; federation between them is at the schema and audit-aggregation layer.

Modus Tertius is the persistent specialist agents within a Secundus instance. Each agent is bound to a specific role and a defined operational scope. Tertius agents have stable identities, capability bindings, and audit trails that persist across sessions.

Modus Quartus is bounded execution: discrete operational tasks with defined inputs, defined outputs, and defined lifetimes. Quartus instances have no memory across invocations; they are non-persistent execution contexts invoked by Tertius agents.

Modus Quintus is ephemeral coordination: short-lived parallel execution patterns invoked for tasks benefiting from parallelism, exploration, or redundancy. Like Quartus, non-persistent; unlike Quartus, exists in coordinated multiplicity that synthesizes into a single result.

4.2 Inheritance Contracts

Each tier inherits a defined subset of the commitments of its tier parent (i.e., the tier immediately above it in the federation hierarchy). Inheritance is one-way and may not weaken higher-tier commitments.

Primus to Secundus. A Secundus instance inherits the full meta-harness specification, federation schema, execution governance specification, and observability commitments. Secundus may bind these to enclave-specific authorization context (populate meta-harness files with enclave content, configure mechanisms for enclave infrastructure) but may not weaken inherited commitments. Strengthening is permitted; weakening is not.

Secundus to Tertius. A Tertius agent inherits a role-bounded subset of the Secundus meta-harness: the mind reasoning posture, the full morals (no narrowing), a subset of the mission scope, bounded memory scope, a subset of the means. The agent contract (Appendix E) specifies the subsetting per agent.

Tertius to Quartus. A Quartus invocation inherits the invoking Tertius agent's authorization for the specific tool or operation being invoked. Quartus has no governance of its own beyond what passes down: means authorization for the specific tool, morals constraints relevant to the operation, audit obligations for the action.

Tertius to Quintus. A Quintus invocation inherits the invoking Tertius agent's authorization for the parallel pattern. Quintus receives a problem statement, a parallelization strategy, and a synthesis policy. Each parallel branch is itself a bounded execution similar to Quartus; the Quintus instance as a whole is the coordinated pattern.

4.3 Delegation Paths

Primus does not delegate operational work. Secundus delegates to Tertius agents through the orchestration layer. Tertius delegates to Quartus or Quintus instances. Quartus and Quintus do not delegate (leaf tiers).

This produces a directed acyclic invocation graph: Secundus invokes Tertius, Tertius invokes Quartus or Quintus, no other paths exist. The acyclic property is architecturally important; it prevents recursive delegation patterns that would make audit and accountability impossible.

4.4 Escalation Paths

Quartus and Quintus escalate to the invoking Tertius agent. Tertius escalates to Secundus. Secundus escalates to Primus or to the enterprise governance function that owns Primus. Direct escalation to Primus is permitted under defined exceptional conditions (typically safety-critical exceptions or detected systemic violations) bypassing intermediate tiers.

The asymmetry between delegation (strictly tier-by-tier) and escalation (tier-by-tier with defined direct paths) is intentional. Delegation is operational; escalation is governance and benefits from the ability to surface critical issues quickly.

4.5 Lifecycle Semantics

Lifecycle progresses from durable to ephemeral as one descends the tier hierarchy.

Primus is domain-durable. Persists for the lifetime of the architectural domain. Revisions are governance acts.

Secundus is enclave-durable. Persists for the lifetime of the enclave's authorization scope. Revisions are configuration acts within the enclave.

Tertius is agent-persistent. Persists from contract approval until retirement. Revisions go through the agent contract review gate.

Quartus is ephemeral. Instantiated by an invocation, exists for the bounded operation, dissolves. Typical lifetime: seconds to minutes.

Quintus is ultra-ephemeral. Instantiated by a parallel pattern invocation, exists for the coordinated execution and synthesis, dissolves.

4.6 The Federation Pattern

Modus Primus uses a hierarchical federation pattern with safety bypasses. Delegation flows strictly through the tier hierarchy. Escalation flows through the tier hierarchy by default with defined direct-escalation conditions for safety-critical issues. Lateral relationships between peer instances at the same tier are not part of the pattern; coordination between peers flows through their common parent.

This pattern was chosen over strict hierarchy (rejected because it makes safety-critical escalation slow) and lattice (rejected because peer relationships introduce coordination complexity not justified for the architectural domain).

4.7 Cross-Enclave Federation

Cross-enclave federation occurs at three layers and never includes direct cross-enclave invocation.

At the Primus layer, federation is *specification-shared*: all Secundus instances inherit from the same Primus, sharing architectural pattern, meta-harness specification, execution governance, and observability commitments.

At the federation schema layer, federation is *artifact-shared*: governance schema registry, audit aggregation patterns, capability registry templates cross enclave boundaries because they are specifications, not operational data.

At the audit aggregation layer, federation is *signal-shared*: aggregate audit signals (conformance reports, schema validation outcomes) cross enclave boundaries because they are aggregate signals, not instance data.

Operational data, agent memory, mission content, identity stores, instance audit records, does not cross. The PAHA enclave constraint applies in full: harness as pattern, instantiated per enclave, with shared governance artifacts only.

4.8 Federation Contract Format

Each Modus tier carries a federation contract: an inheritance section (what it inherits from its parent), a delegation section (what subordinates it may invoke), an escalation section (where it escalates and under what conditions), a lifecycle section (how it persists and is governed), and a conformance section (what verification artifacts it produces).

The contract format is consistent across tiers; tier-specific content varies. Agent contracts (Appendix E) specialize the federation contract format for the Tertius tier.

5. Technical Baseline Management

5.1 WBS-to-Tech-Baseline Relationship

The WBS (Section 3 with Appendix B) is a *specification* of what a Modus Primus instance comprises. The per-enclave tech baseline is the *fulfillment record* of the WBS for a specific enclave. Every entry in the WBS has a corresponding entry in the tech baseline that names the specific product, service, or component fulfilling it. Empty fulfillment slots are gaps; entries in the tech baseline that don't map to a WBS requirement are sprawl.

The WBS is shared across enclaves; tech baselines are per enclave. This is the architectural pattern in PAHA's enclave constraint applied at the SEMP level.

5.2 Per-Enclave Tech Baseline Structure

The per-enclave tech baseline mirrors the WBS structure section-for-section. Same layer numbering, same artifact names, same federation tier breakdown. The content is fulfillment information rather than specification.

For each WBS entry, the tech baseline records: the fulfilling artifact or product (what), the owner or accountable party (who), the maturity status (mature, partial, aspirational), the last review date (when), and any open dependencies or risks.

The full template is in Appendix D.

5.3 Fulfillment Status

Three statuses indicate the maturity of fulfillment.

Mature indicates a real artifact under change control, governed, integrated with downstream enforcement. The fulfillment is operationally trustworthy.

Partial indicates an artifact exists but coverage is incomplete, integration is partial, or governance is not yet fully established. The fulfillment is operationally usable but with caveats.

Aspirational indicates the slot is named in the tech baseline but not yet filled. The fulfillment is paper-only.

The distribution of statuses across the tech baseline is diagnostically useful. A tech baseline dominated by aspirational entries indicates an early-stage instance; a tech baseline with mostly mature entries with some aspirational ones in the runtime assurance layer (Section 9.2 of the WBS) is typical because runtime assurance commitments often lack mature commercial products.

5.4 Tech Baseline Review Cadence

Per-enclave tech baselines are reviewed quarterly at minimum. Sections covering substrate-volatile content (Cognitive Engine, Mechanism Layer) may warrant monthly review during periods of rapid technology change. Reviews surface gaps, identify newly mature fulfillments, and trigger contract revisions where capability changes affect agent contracts.

5.5 Configuration Management for the WBS

Configuration management for the WBS itself follows the enterprise's configuration management practice. [ENTERPRISE: integrate WBS revisions into the enterprise's CM process; if no formal CM practice exists, treat the WBS as a configuration-controlled artifact under this specification's revision discipline.] AI-specific triggers that warrant WBS revision: introduction of a new federation tier, revision of a meta-harness file format, structural change to the federation contract format, addition of a new architectural runtime assurance commitment.

Routine fulfillment updates do not trigger WBS revision; they trigger tech baseline revision.

5.6 Configuration Management for Per-Enclave Tech Baselines

Tech baseline configuration management also follows the enterprise's configuration management practice and applies enclave-by-enclave. Tech baseline revisions are scoped to the enclave; cross-enclave coordination is required only when a tech baseline change affects federation schema conformance.

6. Engineering Specialization

This section specifies the AI-architecture-specific engineering processes. General engineering process elements (configuration management mechanics, broad change control, organizational process governance) are referenced abstractly and are supplied by the enterprise's engineering practice.

6.1 Substrate Volatility and Cognitive Plane Cadence

Foundation models, specialized models, and inference infrastructure evolve on cycles measured in months. Typical engineering cadences for major engineering artifacts (often annual revision review) are insufficient for the cognitive plane.

Specialization. Cognitive plane changes follow a substrate substitution review process (Section 9.4) that is event-triggered rather than calendar-triggered. When a substrate change is proposed, the substrate substitution review evaluates the change, triggers re-verification (Section 7.5), and updates the relevant tech baselines. The cognitive plane review cadence is therefore continuous monitoring with event-driven formal review.

6.2 Agent Contract Authoring and Review

Agent contracts (Appendix E) are authored by agent owners, reviewed at the agent contract review gate (Section 9.5), and registered in the Secundus capability registry. The contract authoring process is specialized because it requires both meta-harness inheritance discipline (correct subsetting of mission and means) and operational role definition (correct specification of authorized invocations).

Specialization. The contract authoring process requires the author to demonstrate, at review, both inheritance correctness (the agent's authorization is a proper subset of Secundus authorization with no weakening of morals) and operational coherence (the agent's role is sufficiently distinct from sibling agents and sufficiently aligned with mission). The review applies both criteria; failure on either prevents contract approval.

6.3 Means Election

A mechanism in the Mechanism Layer becomes a means in means.md only through purposive election. The principle that justifies this discipline is stated in full in Appendix C; this section specifies the engineering process by which election is performed.

Specialization. When a new mechanism is introduced into the Mechanism Layer (a new tool, a new platform integration), it does not automatically become part of the system's disposition. The introducing engineer or product owner must propose the mechanism's election as a means through the means election review (Section 9.6). The review evaluates: whether the mechanism advances the mission, whether its election is consistent with morals, whether the available mechanism inventory justifies the addition, and whether elections are being properly retired when no longer purposive.

This process prevents silent capability creep, the failure mode where every newly procured tool becomes part of the architecture by accretion rather than purposive election.

Worked example. A new code-execution sandbox is procured by the platform team and added to the Mechanism Layer at B.7.1.4. The coder-agent owner proposes its election as a means under the agent's existing mission scope, citing improved isolation and reduced execution overhead. The means election

review (Section 9.6) evaluates the proposal on three axes. First, mission advancement: the proposal demonstrates the new sandbox enables faster iteration on the agent's existing tasks, satisfying mission alignment. Second, morals consistency: the new sandbox provides equivalent or stronger containment compared to the existing means, satisfying constraint preservation. Third, inventory justification: the existing sandbox is retained for legacy workflows but is marked for retirement within two release cycles, satisfying the discipline against unbounded means accretion. The review approves the election; means.md is revised to add the new sandbox under B.3.5.9 (code execution authorization) with a sunset clause for the predecessor. The Mechanism Layer addition required no architectural review; the means election did. That asymmetry is the discipline at work.

6.4 Federation Schema Evolution

The federation schema is shared across enclaves. Schema evolution requires cross-enclave coordination because changes affect every Modus Secundus instance.

Specialization. Federation schema changes go through the federation schema review gate (Section 9.4) which includes representation from each enclave that hosts a Modus Secundus instance. Changes are evaluated for backward compatibility, migration path, and per-enclave impact. Schema versioning is required: at any time, multiple schema versions may be operational across enclaves during migration windows.

6.5 Runtime Assurance Engineering

The runtime assurance commitments (drift detection, mission coherence monitoring, policy deviation detection, explainability surfacing) are architecturally distinctive and require dedicated engineering attention.

Specialization. Runtime assurance is owned by the runtime assurance owner (Section 10.6) and is engineered as an integrated function rather than a collection of monitoring capabilities. The engineering work includes: defining baseline behavior for each instrument, calibrating detection thresholds per enclave, establishing response procedures for each finding type, integrating runtime assurance signals with the V&V evidence base (Section 7), and managing the lifecycle of detection rules as the architecture evolves.

7. Verification and Validation Specialization

This section specifies the AI-architecture-specific V&V approach through four V&V instruments. General V&V methodology is referenced abstractly; enterprises with an existing V&V practice integrate these instruments into it, while enterprises without one treat the instruments as the V&V approach directly.

7.1 Why AI Architecture Requires V&V Specialization

[ENTERPRISE INTEGRATION; Section 7] Several elements in this section may require alignment with the enterprise's existing V&V practice if one exists. Specifically: (a) V&V terminology, the enterprise may use specific terminology that differs from generic INCOSE *verification* and *validation*; substitute as needed. (b) Quantitative bounds for V&V findings (acceptable drift rates, acceptable coherence variance, acceptable deviation rates), these are calibrated to the enterprise's risk tolerance and are specified per enclave in the tech baseline (Appendix D). (c) Stakeholder feedback methodology in Section 7.3, align to whatever method the enterprise uses for collecting validation evidence (surveys, structured interviews, after-action reviews). (d) Change control gates for substrate substitution in Section 7.5, align to the enterprise's change control process if one exists. Search for [ENTERPRISE: markers in this section to find specific substitution points.

Three properties motivate specialization beyond general V&V approaches.

Substrate volatility means the system being verified may have its underlying machinery replaced between verifications. Generic V&V verifies a stable system; AI V&V must verify under substrate change.

Behavioral non-determinism means the same input may produce different outputs across invocations within bounded variation. Generic V&V verifies output identity within tolerance; AI V&V verifies output behavior within an envelope.

Continuous mission alignment means the system can become misaligned with its mission through gradual drift even when its specification is unchanged. Generic V&V validates fitness at milestones; AI V&V monitors fitness continuously. [ENTERPRISE: confirm V&V terminology, *verification* and *validation* are used per generic INCOSE convention; substitute enterprise-specific terms if they differ.]

7.2 Verification of WBS Conformance

The first V&V instrument verifies that a Modus instance conforms to the architectural specification stated in the WBS. Methods: static review of artifacts, schema validation, trace verification of inheritance preservation. Performed at instantiation, at revision, and periodically during operational lifetime.

A Modus instance passes WBS conformance verification when all required artifacts are present and conform to format, schema validation passes for all schema-conformant artifacts, inheritance trace verification confirms no weakening of higher-tier commitments, and the reviewer issues a conformance certificate that becomes part of the configuration management record.

7.3 Validation of Mission Alignment

The second V&V instrument validates that the Modus instance's runtime behavior is aligned with its declared mission. Methods: continuous mission coherence monitoring, periodic outcome review against mission objectives, stakeholder feedback from human users and downstream consumers.

A Modus instance is validated as mission-aligned when continuous mission coherence monitoring shows no sustained drift, outcome review confirms the federation is meeting success criteria, and stakeholder feedback indicates the federation is fit for its intended use. Failure on any of the three triggers a validation finding requiring resolution before the next operational period. [ENTERPRISE: align stakeholder feedback methodology to the enterprise's specified approach for collecting validation evidence from operational stakeholders.]

7.4 Runtime Assurance as Continuous Validation Evidence

The third V&V instrument elevates the runtime assurance commitments (Section 9.2 of the WBS) into a V&V instrument. Drift detection, mission coherence monitoring, policy deviation detection, and explainability surfacing together produce a continuous evidence base that the Modus instance is operating within its specification and toward its mission.

This is the most distinctive specialization in this specification. Generic V&V treats runtime monitoring as observability; AI V&V treats it as continuous validation. The runtime assurance evidence base is the primary V&V input between formal review milestones. [ENTERPRISE: specific quantitative bounds for acceptable drift rates, coherence variance, and policy deviation rates are calibrated per enclave in the tech baseline (Appendix D); the enterprise's risk tolerance constrains those calibrations.]

7.5 Substrate Substitution Re-Verification

The fourth V&V instrument addresses substrate volatility. When a cognitive substrate changes, the Modus instance is re-verified before the change takes effect.

Methods: behavioral envelope testing (verifying outputs within the envelope established by the prior substrate), inheritance preservation re-verification (confirming the inheritance chain still honors commitments), runtime assurance baseline re-establishment, stakeholder re-validation for major substrate changes.

The depth of re-verification is calibrated to the magnitude of the change. Minor version updates require lightweight re-verification; major substrate replacements require comprehensive re-verification. [ENTERPRISE: substrate substitution is a configuration change governed by the enterprise's change control process if one exists; align the substrate substitution review gate to those gates where applicable.]

7.6 The V&V Evidence Base

The four V&V instruments together produce a continuous evidence base structured for review. At instantiation, the evidence comprises WBS conformance verification, initial mission alignment validation, baseline runtime assurance, and substrate qualification. During operation, the evidence is continuously updated by runtime assurance and reviewed periodically. At revision, the relevant evidence subset is regenerated to support the revision review. At retirement, the historical evidence record is preserved per audit retention.

The structured evidence base is what makes the V&V approach reviewable rather than ad hoc.

8. Domain-Specific Risk Management

This section enumerates the AI-architecture-specific risk categories. Enterprises with an existing risk management process integrate these categories into it; enterprises without one should treat the AI-specific categories as part of a broader risk management practice they will need to maintain.

Substrate risks. Vendor volatility, capability degradation, deprecation of foundation models, license model changes, geographic availability constraints affecting GovCloud or classified deployments. These risks affect the Cognitive Engine and propagate to mind.md elections. Mitigation depends on substrate adapter portability and on maintaining substrate diversity where operationally feasible.

Federation risks. Schema drift across enclaves, ATO scope divergence between Secundus instances, partner enclave contract changes affecting federation participation, classification boundary changes affecting which schemas can cross. Mitigation is the federation schema review process (Section 9.4) and per-enclave configuration management.

Means-election risks. Unelected mechanism sprawl (mechanisms that accumulate in the Mechanism Layer without being purposively elected), accreted capability creep (mechanisms elected as means without retirement of superseded means), election drift (means.md no longer reflecting actual operational practice). Mitigation is the means election review (Section 9.6) and periodic means.md audit.

Agent contract risks. Role drift (agent runtime behavior diverging from declared role), permission creep (agent contract revisions that incrementally broaden authorization without explicit justification), lifecycle gaps (retired agents whose audit obligations are not properly transferred). Mitigation is the agent contract review (Section 9.5) and runtime assurance drift detection.

Runtime assurance risks. False confidence in monitoring coverage (assuming runtime assurance catches what it does not), undetected deviation (deviations that fall below detection thresholds or that the assurance instruments are not configured to catch), monitoring blind spots (categories of behavior that the runtime assurance layer does not address). Mitigation is the runtime assurance owner's responsibility and includes periodic blind-spot analysis.

These categories are not exhaustive; they are the AI-architecture-specific specializations. General risk categories (program risk, technical risk, operational risk, security risk) apply additionally and are managed under whatever risk management practice the enterprise maintains.

9. Domain-Specific Technical Reviews

This section specifies the AI-architecture-specific review gates. General technical review structures (preliminary design reviews, critical design reviews, operational readiness reviews) are referenced abstractly and are supplied by the enterprise's engineering practice; the AI-specific gates feed evidence into those broader reviews where they exist.

[ENTERPRISE INTEGRATION; Section 9] The seven review gates specified below are AI-architecture-specific specializations. Several integrations may be required depending on the enterprise's engineering practice: (a) name each review gate using the enterprise's gate-naming convention where one exists. (b) Specify the relationship between each AI-specific gate and the enterprise's broader reviews (preliminary design review, critical design review, operational readiness review, etc.), the AI-specific gates feed evidence into the broader reviews; the enterprise may have an existing integration pattern. (c) Cadence specifications below default to "as needed" or "quarterly minimum"; confirm against the enterprise's review schedule. (d) Participant roles below should be cross-referenced to specific named roles in the enterprise's organizational baseline.

9.1 WBS Architectural Review

Reviews proposed changes to the WBS itself: structural changes to the meta-harness layer, new federation tier definitions, new architectural runtime assurance commitments, format changes to the federation contract.

Entry criteria: a written change proposal with rationale, impact analysis across enclaves, and conformance impact assessment. Exit criteria: approval or rejection with documented reasoning, integration into the WBS configuration management record. Cadence: as needed, infrequent. Participants: WBS owner, runtime assurance owner, federation coordinator, governance representatives.

9.2 Per-Enclave Tech Baseline Review

Reviews tech baseline updates for a specific enclave: fulfillment changes, status transitions (aspirational to partial to mature), gap surfacing, and configuration management entries.

Entry criteria: tech baseline revision proposal, fulfillment evidence for status changes. Exit criteria: approved tech baseline revision integrated into enclave configuration management. Cadence: quarterly minimum, more frequent for substrate-volatile sections. Participants: enclave tech baseline owner, relevant capability owners, runtime assurance representative.

9.3 Modus Instance Certification Review

Reviews readiness for a new Modus Secundus instance to enter operation, or for a new Tertius agent to be registered in an existing Secundus capability registry.

Entry criteria: complete federation contract or agent contract, V&V evidence base for the new instance (WBS conformance verification, initial mission alignment validation, baseline runtime assurance, substrate qualification), tech baseline fulfillment for required artifacts. Exit criteria: certification decision, integration into the relevant capability registry. Cadence: as needed. Participants: Primus governance authority, instance owner, V&V representative.

9.4 Federation Schema Review

Reviews proposed changes to the federation schema: governance schema registry updates, audit aggregation pattern changes, capability registry template revisions.

Entry criteria: schema change proposal, backward compatibility analysis, per-enclave impact assessment, migration plan. Exit criteria: approved schema revision with migration window specified, configuration management record updated. Cadence: as needed. Participants: federation coordinator, representatives from each Modus Secundus enclave, governance authority.

9.5 Agent Contract Review

Reviews new or revised agent contracts: identity, inheritance, delegation, escalation, lifecycle, conformance sections all evaluated.

Entry criteria: complete agent contract, inheritance correctness analysis, operational coherence justification, conformance plan. Exit criteria: approved agent contract registered in Secundus capability registry, or rejection with documented findings. Cadence: as needed. Participants: agent owner, Secundus orchestration owner, V&V representative.

9.6 Substrate Substitution Review

Reviews proposed substrate changes in the Cognitive Engine: foundation model replacement, routing policy revision, multi-substrate transitions.

Entry criteria: substrate change proposal, behavioral envelope analysis, inheritance preservation analysis, runtime assurance impact assessment. Exit criteria: approval or rejection, substrate substitution re-verification (Section 7.5) if approved, configuration management record updated. Cadence: as needed, event-driven. Participants: cognitive plane owner, runtime assurance owner, V&V representative.

9.7 Means Election Review

Reviews proposed means.md elections (introducing a new means or retiring an existing means).

Entry criteria: election proposal with rationale tying the means to mission advancement, morals consistency analysis, mechanism availability confirmation. Exit criteria: approved election with means.md revision, or rejection. Cadence: as needed. Participants: means.md owner, mission stakeholder representative, governance representative.

The seven review gates above specialize general technical review structures. Where an enterprise maintains broader reviews (preliminary design review, critical design review, operational readiness review, etc.), they apply additionally and consume V&V evidence from the AI-architecture-specific gates as input.

10. Domain-Specific Organizational Responsibilities

This section specifies the AI-architecture-specific roles required for engineering practice. General organizational baseline elements (reporting lines, named roles, authority boundaries) are referenced abstractly and are supplied by the enterprise's organizational structure.

[ENTERPRISE INTEGRATION; Section 10] Role definitions below specify AI-architecture-specific responsibilities. Reporting lines, specific named roles, and authority boundaries must align to the enterprise's organizational baseline. Substitute enterprise-specific role designations where they exist; the role descriptions below are intentionally generic to permit alignment.

WBS owner. Canonical authority for the architectural specification. Approves WBS revisions, chairs the WBS architectural review, maintains the configuration management record for the WBS itself. Typically a senior enterprise architect within the Governance and Strategy Directorate.

Per-enclave tech baseline owners. One per enclave. Maintain the fulfillment record for the enclave's WBS instantiation. Approve tech baseline revisions, chair the per-enclave tech baseline review, surface gaps to the WBS owner when fulfillment indicates specification problems.

Five-M file owners. Each meta-harness file has a natural ownership home. `mind.md` is owned by the epistemic authority for the AI architecture domain (typically a chief AI architect or equivalent). `morals.md` is owned by the governance and compliance function. `mission.md` is owned by program leadership for the relevant mission scope. `memory.md` is owned by data governance. `means.md` is owned by the capability owner aggregate (the function that aggregates per-mechanism owners). Each owner approves revisions to their file and represents the file in cross-cutting reviews.

Mechanism Layer owner. Maintains the catalog of operationally available mechanisms across the enterprise. Coordinates with capability owners for individual mechanisms. Triggers means election reviews when new mechanisms are introduced.

Federation coordinator. Maintains the federation schema. Coordinates cross-enclave schema changes through the federation schema review. Aggregates federation-level audit signals and provides them to the V&V evidence base. Acts as the bridge between Primus-level governance and per-enclave Secundus operations.

Runtime assurance owner. Engineers and operates the architectural runtime assurance instruments (drift detection, mission coherence monitoring, policy deviation detection, explainability surfacing). Defines

baselines, calibrates thresholds, manages detection rules. Provides runtime assurance evidence to the V&V evidence base. Reports to both the engineering function (for technical operation) and the governance function (for findings escalation).

Agent owners. Each agent contract has an accountable owner. Approves agent contract revisions, monitors agent runtime behavior, initiates retirement when appropriate. Owners are typically named roles within the function the agent serves rather than individuals.

V&V function. Performs the four V&V instruments specified in Section 7. Maintains the V&V evidence base. Provides certification recommendations for the Modus instance certification review. The V&V function integrates with the enterprise's existing V&V organization where one exists, applying AI-specific specialization on top of general V&V practice.

The roles above operate within the enterprise's organizational structure. Specific personnel assignments, reporting lines, and authority boundaries are supplied by the enterprise's organizational baseline.

References

References use Harvard-style citations with hanging indents. The list below covers works cited in the body and appendices.

INCOSE (2023). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th edition. International Council on Systems Engineering, Hoboken, NJ: Wiley.

ISO/IEC/IEEE (2022). *ISO/IEC/IEEE 42010:2022, Software, systems and enterprise, Architecture description*. International Organization for Standardization, Geneva.

Longmire, J. (2026). *Portable Agent Harness Architecture: A Capability-Centric Framework for Governed AI Ecosystems in Sovereignty-Bounded Enterprises*, Revision 2.2. Zenodo.
<https://doi.org/10.5281/zenodo.20112631>

The Open Group (2022). *TOGAF Standard, 10th Edition*. The Open Group, San Francisco.

Appendix A - Referenced Architectural Framework (PAHA)

Portable Agent Harness Architecture (PAHA) is the architectural framework that this specification implements. The PAHA specification is a separate document (Longmire 2026, Revision 2.2; doi:10.5281/zenodo.20112631) and is not reproduced here. The elements relevant to this specification are summarized below.

PAHA establishes that enterprises operating under sovereignty constraints, multi-vendor obligations, or cross-enclave security boundaries require an architectural pattern in which the durable architectural layer is a meta-harness that provides centralized governance, bounded execution, and substrate arbitration over fit-for-purpose operational consoles and composable agents.

The PAHA architectural primitives, governance contracts, execution boundaries, substrate adapters, capability registries, map to Modus Primus artifacts. The PAHA three-plane decomposition (governance, cognitive, operational) maps to the WBS layer structure. The PAHA seven minimum viable harness services map to specific Modus Primus components. The PAHA enclave constraint maps to the Modus federation pattern.

Where this specification makes architectural commitments, it does so in PAHA-conformant terms. Readers needing the full PAHA specification, including the three reference figures (three architectural planes, meta-harness with consoles and agents, deployment view across enclaves), should consult the PAHA paper directly.

Appendix B - Detailed Technical Baseline Specification

This appendix provides the artifact-level specification (Work Breakdown Structure) for a Modus Primus instance. Section 3 of this specification provides the layer overview; this appendix specifies each artifact and its content scope.

The WBS preserves the five-M meta-harness vocabulary, the five Modus tier model, the means/mechanisms distinction (Appendix C), and the layered architecture. Cross-reference annotations indicate where artifacts depend on or are governed by other artifacts.

B.1 Layer Boundaries

Before the WBS itself, the boundaries between three layers are stated explicitly because several sections would otherwise drift across them.

The meta-harness layer (B.3) governs disposition. What the system is for, how it reasons, what it must not do, what it remembers, and what capabilities it has. These are stable, declarative properties of the system as a whole. They change on cycles measured in months or years.

The orchestration layer (B.5) governs decomposition and sequencing. Given a task and a meta-harness disposition, how is the task broken into sub-tasks, which agents are invoked, in what order, and how are partial results combined. Orchestration is mechanical given the meta-harness; it does not introduce new policy.

The agent layer (B.6) governs role-bound execution. Each agent is a bounded persona with a specific role, scoped capabilities, and a delegated subset of meta-harness authority. Agents do not redefine the meta-harness; they operate within it.

The rule that resolves overlap: meta-harness defines what is permissible and required; orchestration defines what is sequenced and combined; agents perform the work within both constraints. When in doubt, the file or service belongs to the layer that defines the policy, not the layer that applies it.

B.2 System Orientation

The orientation layer establishes system identity and federation posture. Read at boot or session start; stable across runtime.

B.2.1 mode.md

- B.2.1.1 System identity
- B.2.1.2 Governing philosophy
- B.2.1.3 Meta-harness declaration
- B.2.1.4 Operational orientation
- B.2.1.5 Constraint inheritance model
- B.2.1.6 Federated hierarchy declaration (*refers to B.8*)
- B.2.1.7 Execution doctrine (*refers to B.7*)
- B.2.1.8 Admissibility principles (*refers to B.3*)
- B.2.1.9 Delegation philosophy (*refers to B.5*)
- B.2.1.10 Root orchestration posture (*refers to B.5*)

Cross-references make explicit that mode.md declares posture and points to the layers that specify mechanics.

B.3 Meta-Harness Layer

The five-M decomposition. Each file specifies one durable concern of the system's disposition. Files are declarative, not procedural.

B.3.1 mind.md, How the system reasons

- B.3.1.1 Epistemic posture
- B.3.1.2 Reasoning frameworks
- B.3.1.3 Inferential rules
- B.3.1.4 Cognitive discipline
- B.3.1.5 Truth prioritization
- B.3.1.6 Uncertainty handling

- B.3.1.7 Context interpretation
- B.3.1.8 Analytical methods
- B.3.1.9 Decision heuristics
- B.3.1.10 Model orientation rules

B.3.2 morals.md, What the system must not do

- B.3.2.1 Ethical constraints
- B.3.2.2 Prohibited actions
- B.3.2.3 Safety boundaries
- B.3.2.4 Human oversight requirements
- B.3.2.5 Escalation requirements (*when escalation must be invoked, as principle; runtime triggers in B.7.1*)
- B.3.2.6 Delegation limits
- B.3.2.7 Applicable compliance regimes
- B.3.2.8 Authorization principles (*declarative; runtime gate in B.7.1*)
- B.3.2.9 Execution restrictions
- B.3.2.10 Admissibility conditions

"Policy enforcement" was removed from this file in Pass 1, it is mechanism (B.7.2), not policy. Morals.md declares the constraints; execution-runtime.md enforces them.

B.3.3 mission.md, What the system is for

- B.3.3.1 Mission statement
- B.3.3.2 Operational objectives
- B.3.3.3 Priority hierarchy
- B.3.3.4 Success criteria
- B.3.3.5 Domain alignment
- B.3.3.6 Task admissibility (*refers to B.3.2*)
- B.3.3.7 Strategic intent
- B.3.3.8 Outcome evaluation
- B.3.3.9 Delegation goals
- B.3.3.10 Mission inheritance rules (*refers to B.8*)

B.3.4 memory.md, What the system remembers and forgets

- B.3.4.1 Context retention
- B.3.4.2 Persistence policies
- B.3.4.3 Retrieval rules

- B.3.4.4 Session continuity
- B.3.4.5 Knowledge grounding
- B.3.4.6 State management
- B.3.4.7 Memory boundaries
- B.3.4.8 Temporal relevance
- B.3.4.9 Audit history retention (*declarative; audit storage in B.9*)
- B.3.4.10 Recall prioritization

B.3.5 means.md, The system's purposive repertoire for advancing its mission within its morals

- B.3.5.1 Capability inventory (*collapsed in Pass 2 from three near-synonymous entries; see Appendix C for the means/mechanisms distinction*)
- B.3.5.2 Workflow capabilities
- B.3.5.3 Automation permissions
- B.3.5.4 Retrieval systems (*declarative; tool integration in B.6*)
- B.3.5.5 Execution pathways (*declarative; orchestration mechanics in B.5*)
- B.3.5.6 Communication channels (*declarative; tool integration in B.6*)
- B.3.5.7 Media generation
- B.3.5.8 Analysis capabilities
- B.3.5.9 Code execution authorization (*declarative; execution gate in B.7*)
- B.3.5.10 Human interaction surfaces
- B.3.5.11 Capability inheritance rules (*refers to B.8*)
- B.3.5.12 Capability availability matrix

means.md was significantly tightened in Pass 1 and Pass 2. Items in the original draft that turned out to be mechanism (tool integrations, API surfaces, external system interfaces) were moved to B.6. Items that were near-synonyms (capability definitions, skills inventory, feature mappings) were collapsed. The remaining items are all purposive declarations of the system's repertoire toward its mission. See Appendix C for the principle that justifies this discipline.

B.4 Cognitive Engine Layer

The cognitive engine is the substrate over which agents reason. mind.md elects reasoning capabilities from this layer (selection relation per Appendix C).

B.4.1 model/

- B.4.1.1 Foundation models
- B.4.1.2 Specialized models
- B.4.1.3 Multi-model routing

- B.4.1.4 Inference management
- B.4.1.5 Context windows
- B.4.1.6 Embedding systems
- B.4.1.7 Fine-tuned variants
- B.4.1.8 Reasoning modes (*mechanics; reasoning policy in B.3.1*)
- B.4.1.9 Latency optimization
- B.4.1.10 Confidence handling (*mechanics; uncertainty policy in B.3.1*)

The cognitive engine implements reasoning and confidence handling per the policy declared in mind.md. The engine does not redefine reasoning; it executes it.

B.5 Orchestration Layer

Orchestration sequences and decomposes. It does not introduce new policy. mission.md and the agent contracts elect orchestration patterns from this layer.

B.5.1 orchestration/

- B.5.1.1 orchestrator.md, Top-level orchestration entry point
- B.5.1.2 routing.md, Which agent receives which task
- B.5.1.3 delegation.md, How tasks pass to subordinate Modus tiers
- B.5.1.4 sequencing.md, Order of operations
- B.5.1.5 coordination.md, Multi-agent coordination patterns
- B.5.1.6 arbitration.md, Resolving competing agent outputs
- B.5.1.7 load-balancing.md, Distribution across substrate
- B.5.1.8 task-decomposition.md, Breaking tasks into sub-tasks
- B.5.1.9 execution-planning.md, Sequencing under constraint
- B.5.1.10 recovery-handling.md, Failure modes and retry semantics

Orchestration mechanics only. Reasoning quality, ethical constraint, and admissibility live in the meta-harness. When orchestration appears to require a policy decision, that decision belongs upstream in mind.md or morals.md.

B.6 Agent Layer

Agents are role-bound personas operating within meta-harness disposition and orchestration sequencing. See Appendix E for the agent contract format.

B.6.1 agents/

- B.6.1.1 agents.md, Catalog and contract specification
- B.6.1.2 researcher-agent.md

- B.6.1.3 coder-agent.md
- B.6.1.4 reviewer-agent.md
- B.6.1.5 planner-agent.md
- B.6.1.6 analyst-agent.md
- B.6.1.7 mission-agent.md
- B.6.1.8 governance-agent.md
- B.6.1.9 retrieval-agent.md
- B.6.1.10 synthesis-agent.md

B.6.2 agent-mechanics/

- B.6.2.1 Agent lifecycle
- B.6.2.2 Agent identity
- B.6.2.3 Agent memory scope (*mechanics; policy in B.3.4*)
- B.6.2.4 Multi-agent coordination (*mechanics; orchestration in B.5*)
- B.6.2.5 Swarm execution (*mechanics; orchestration in B.5*)

B.6.3 agent-contract.md (*collapsed in Pass 2 from a directory of cross-references*)

Declares the agent's inheritance points as cross-references plus any agent-specific governance properties:

- Role inheritance (*refers to B.3.3*)
- Permission inheritance (*refers to B.3.5 and B.3.2*)
- Context scoping (*refers to B.3.4*)
- Delegation limits (*refers to B.3.2*)
- Escalation rules (*refers to B.3.2 and B.7*)
- Swarm governance (*refers to B.8*)

Agent governance properties are mostly references back to meta-harness files. Agents inherit their governance from the meta-harness; agent-specific governance is rare and should be explicitly justified when it appears.

B.7 Mechanism Layer

The tools, integrations, and external systems that agents invoke. Means are purposive (B.3.5); mechanisms are operationally agnostic catalog. The selection relation between means and mechanisms is specified in Appendix C.

B.7.1 tools/

- B.7.1.1 API integrations
- B.7.1.2 Search systems

- B.7.1.3 File systems
- B.7.1.4 Code execution environments
- B.7.1.5 Communications channels
- B.7.1.6 Data analysis pipelines
- B.7.1.7 Scheduling systems
- B.7.1.8 Visualization systems
- B.7.1.9 Simulation systems
- B.7.1.10 Automation pipelines

Renamed from "Means Layer" in Pass 2 to disambiguate from means.md while preserving the five-M aesthetic in the meta-harness.

B.8 Execution Governance Layer

Split in Pass 1 into two artifacts: execution-policy.md (governance-facing, declarative) and execution-runtime.md (operational-facing, mechanical).

Audit lifecycle. morals.md (B.3.2) and execution-policy.md (B.8.1) *declare* what must be audited; execution-runtime.md (B.8.2) *emits* audit records; observability (B.10) *stores and queries* them; memory.md (B.3.4.9) *declares retention* policy for the resulting records. The four touchpoints are ordered stages of a single lifecycle, not duplicate concerns.

B.8.1 execution-policy.md, Governance-facing

Declares what must be enforced. Policy specification, not enforcement mechanics. Read by agents and orchestration before action.

- B.8.1.1 Validation rules
- B.8.1.2 Human approval gates (*declarative; policy specification*)
- B.8.1.3 Safety verification requirements
- B.8.1.4 Policy enforcement specification
- B.8.1.5 Risk assessment criteria
- B.8.1.6 Real-world action authorization (*declarative; runtime gate in B.8.2*)
- B.8.1.7 Escalation triggers (*runtime criteria; requirements in B.3.2.5*)
- B.8.1.8 Approval workflow specifications
- B.8.1.9 Audit logging requirements (*what must be logged; storage in B.10*)
- B.8.1.10 Required compliance artifacts

B.8.2 execution-runtime.md, Operational-facing

Specifies how policy is enforced at runtime. Mechanism, not policy. Read by the runtime engine.

- B.8.2.1 Pre-action validation gates
- B.8.2.2 Approval-workflow runtime
- B.8.2.3 Transaction controls
- B.8.2.4 Rollback procedures
- B.8.2.5 Environment isolation
- B.8.2.6 Sandbox enforcement
- B.8.2.7 Resource limit enforcement
- B.8.2.8 Real-world action gating
- B.8.2.9 Failure containment
- B.8.2.10 Runtime audit emission (*mechanism; storage and analysis in B.10*)

The split mirrors the PAHA distinction between governance plane policy (declarative obligations) and operational plane enforcement (runtime mechanisms). It also clarifies who reads what: agents and orchestration read execution-policy.md to know what is permitted; the runtime engine reads execution-runtime.md to know how to enforce. This split is also an instance of the means/mechanisms principle applied within the execution governance layer.

B.9 Federated Architecture Layer

The Modus tier model. Section 4 of this specification specifies the federation mechanics in operational detail (inheritance contracts, delegation paths, escalation paths, lifecycle semantics, federation pattern, cross-enclave federation, federation contract format). This appendix entry establishes the tier vocabulary.

- B.9.1 Modus Primus, Root governance (domain-durable)
- B.9.2 Modus Secundus, Domain orchestration (enclave-durable)
- B.9.3 Modus Tertius, Federated specialists (agent-persistent)
- B.9.4 Modus Quartus, Tool-bound execution (ephemeral)
- B.9.5 Modus Quintus, Ephemeral / swarm execution (ultra-ephemeral)

B.10 Observability and Assurance Layer

Rebalanced in Pass 1 to distinguish standard observability infrastructure from architecturally distinctive runtime assurance commitments.

B.10.1 Standard observability infrastructure

Necessary but not architecturally distinctive. Every mature observability framework carries these.

- B.10.1.1 Telemetry collection
- B.10.1.2 Auditability (*storage and query of audit events*)
- B.10.1.3 Traceability (*linking actions across layers*)

- B.10.1.4 Compliance artifact collection
- B.10.1.5 Behavioral analytics

B.10.2 Architectural runtime assurance

Architecturally distinctive. These commit the framework to runtime monitoring of properties that most observability frameworks do not treat as first-class. Section 7.4 of this specification specifies how these instruments serve as continuous V&V evidence.

- **B.10.2.1 Drift detection.** Monitoring whether agent behavior diverges from declared mission and morals. Distinct from standard model drift; treats drift as a governance property, not a model-quality property.
- **B.10.2.2 Mission coherence monitoring.** Runtime verification that agent action sequences are advancing the declared mission rather than drifting into adjacent or contradictory work. Rare in observability frameworks; an architectural choice that distinguishes Modus Primus from generic agent platforms.
- **B.10.2.3 Policy deviation detection.** Real-time detection of agent or orchestration behavior that violates declared policy from execution-policy.md (B.8.1). Distinct from after-the-fact audit because deviation detection is intended to halt execution before the deviation is consummated.
- **B.10.2.4 Explainability surfacing.** Real-time provision of decision rationale to human operators. Distinct from after-the-fact audit because it operates in the runtime control loop, not the analysis loop.

The split between B.10.1 and B.10.2 makes the architectural commitments visible. A reviewer scanning the original draft might have read all entries as standard observability material; the rebalanced structure distinguishes what is necessary infrastructure from what is a design choice worth defending.

B.11 Cross-Layer Inheritance Summary

For each layer, the WBS specifies inheritance points that propagate purposive declarations from the meta-harness into operational behavior:

- mind.md (B.3.1) elects from Cognitive Engine (B.4)
- morals.md (B.3.2) + execution-policy.md (B.8.1) elect from execution-runtime.md (B.8.2)
- mission.md (B.3.3) elects from Orchestration (B.5) jointly with agent contracts
- memory.md (B.3.4) elects from Observability infrastructure (B.10)
- means.md (B.3.5) elects from Mechanism Layer (B.7)

The selection relation is specified in Appendix C and is the foundational architectural principle that organizes the WBS.

Appendix C - Architectural Principle: Means and Mechanisms

This appendix states the means/mechanisms distinction in full. Section 2 of this specification summarizes it; this appendix provides the complete principle as a reference for architectural decisions.

C.1 The Distinction

A *means* is purposive. It is what the system uses *toward* its mission within its morals. Means carry teleological weight: their existence is justified by what they advance, not by what they can do in isolation.

A *mechanism* is operationally agnostic. It is the machinery by which something works, independent of why. Mechanisms exist as available capacity. They become meaningful when elected by some purposive layer; until then they are inventory.

Means and mechanisms describe the same underlying reality from two different stances. The same code execution environment may be inventoried as a mechanism (the Mechanism Layer in B.7) and elected as a means (means.md in B.3.5). The first description tells what is available; the second tells what is purposively in use.

The assistant-centric pattern that PAHA critiques conflates means and mechanisms: a vendor copilot ships a bundled set of mechanisms (its tools, its memory, its retrieval) and presents them as the system's means by default, without requiring purposive election. The capability-centric pattern that PAHA proposes separates them: mechanisms are inventoried operationally and elected as means through purposive declaration in the meta-harness.

C.2 The Selection Relation

For each purposive declaration in the meta-harness layer, there is a corresponding mechanism layer that catalogs operationally available capacity. The relationship between them is *selection*, not *abstraction*. The purposive layer elects from the mechanism layer; the mechanism layer does not generate the purposive layer.

Purposive declaration	Selects from	Mechanism layer
mind.md (B.3.1)	reasoning machinery	Cognitive Engine (B.4)
morals.md + execution-policy.md (B.3.2 + B.8.1)	enforcement machinery	execution-runtime.md (B.8.2)
mission.md (B.3.3)	sequencing machinery	Orchestration (B.5)
memory.md (B.3.4)	retention machinery	Observability infrastructure (B.10)
means.md (B.3.5)	operational machinery	Mechanism Layer (B.7)

The pattern is consistent across the architecture: a purposive layer elects from a mechanism layer. Mechanisms not elected by any purposive layer are not part of the system's disposition even if they are operationally available. This is the architectural commitment that makes the framework governable: the question of what the system uses (mechanism inventory) is separable from the question of what the system is for (purposive disposition).

C.3 Consequences for the Architecture

The means/mechanisms distinction has four consequences that propagate through the WBS.

It justifies the meta-harness layer's declarative character. The meta-harness files (mind, morals, mission, memory, means) are purposive declarations. They specify how the system reasons, what it must not do, what it is for, what it remembers across time, and what it uses toward those ends. They do not specify how those declarations are mechanically realized; that is the mechanism layer's job. A meta-harness file that drifts into mechanism description has stopped being a purposive declaration.

It justifies the splits within layers. The execution governance layer is split into `execution-policy.md` (governance-facing, declarative) and `execution-runtime.md` (operational-facing, mechanical). This split is an instance of the means/mechanisms principle applied within a single layer: the policy file declares what must be enforced; the runtime file specifies how enforcement is performed. The same pattern appears elsewhere in the WBS: agent contracts (B.6.3) declare role-bound authority; agent mechanics (B.6.2) specify how agent lifecycle and identity are operationally managed.

It justifies the discipline against capability sprawl. A new tool in the Mechanism Layer does not automatically become part of the system's disposition. It becomes a means only through purposive election in `means.md`. This is the means election review (Section 9.7 of this specification) and the discipline that prevents silent capability creep. Without the principle, the architecture accumulates capability through accretion; with the principle, capability is added through deliberate election.

It justifies the federation pattern. The federation contract format (Section 4.8) inherits the means/mechanisms structure. A Modus Secundus instance inherits the meta-harness specification (purposive declarations) from Modus Primus and binds them to enclave-specific mechanism inventories. The purposive declarations propagate through the federation; the mechanism inventories are local to each enclave. This is the structural property that makes the federation governable across enclaves while preserving operational separation between them.

C.4 The Test for Future Specification

When adding to the architecture, ask: *is this a purposive declaration or a mechanism catalog?*

If purposive, it belongs in the meta-harness layer or one of its declarative cousins (`execution-policy.md`, `agent-contract.md`). It should describe what the system uses *toward* its mission, not what is available.

If mechanism, it belongs in one of the mechanism layers (Cognitive Engine, Orchestration, Mechanism Layer, execution-runtime.md). It should describe what is operationally available, not what is purposively elected.

When in doubt, the test is whether the entry would change if the mission changed. Purposive declarations would; mechanism catalogs would not. A new tool integration is invariant to the mission; it is mechanism. A new operational objective is sensitive to the mission; it is purposive.

This test is sufficient to resolve most ambiguity. The remaining cases are typically composite: an artifact that contains both purposive declarations and mechanism descriptions. The resolution is to split the artifact along the principle, as was done with execution-policy.md and execution-runtime.md.

Appendix D - Per-Enclave Tech Baseline Template

The template that each enclave populates to record fulfillment of the WBS for its specific operational context.

For each WBS section, the tech baseline records:

Field	Content
WBS reference	Section number and artifact name from the WBS
Fulfilling artifact	Specific product, service, or component fulfilling the requirement
Owner	Accountable party (role, not individual)
Maturity status	Mature / Partial / Aspirational
Last review date	Date of most recent fulfillment review
Open dependencies	Dependencies on other tech baseline entries or external factors
Open risks	Specific risks affecting the fulfillment

The template is populated section-by-section, mirroring the WBS structure. The complete template covers all WBS sections including the meta-harness files, the mechanism layers, the execution governance layer, the federation contracts, and the observability infrastructure.

Per-enclave tech baselines are maintained under the enterprise's configuration management process and reviewed at the per-enclave tech baseline review (Section 9.2).

Appendix E - Agent Contract Format Specification

The format that every Modus Tertius agent contract conforms to. Each persistent agent (researcher-agent.md, coder-agent.md, etc.) is authored against this format.

Section 1: Identity

Agent name (unique within Secundus), agent role (one-sentence description), tier (always Modus Tertius), owner (accountable role), version and revision history, audit identity.

Section 2: Inheritance

Mind binding (Secundus reasoning posture with role-specific specializations if any), morals inheritance (full Secundus morals plus role-specific strengthenings if any), mission scope (subset of Secundus mission), memory scope (bounded by role and classification), means authorization (subset of Secundus means), federation schema obligations.

Section 3: Delegation

Authorized Quartus invocations (specific tool operations with conditions and constraints), authorized Quintus invocations (parallel patterns with synthesis policies and resource bounds), inter-agent invocation pattern (which other Tertius agents may be requested through Secundus orchestration), delegation governance context (constraints passed down to invocations).

Section 4: Escalation

Routine escalation triggers (conditions for surfacing to Secundus), direct escalation triggers (defined exceptional conditions for bypassing Secundus), escalation audit requirements, failure modes for unhandled escalation.

Section 5: Lifecycle

Instantiation conditions, operational persistence model, revision triggers, retirement triggers, post-retirement obligations.

Section 6: Conformance

Audit trail completeness criteria, runtime assurance signals (which signals at what frequency under what conditions), federation schema conformance reports, inheritance preservation evidence requirements, anomaly response evidence requirements.

A complete agent contract is typically 3-6 pages of markdown. Shorter contracts are usually under-specified; longer contracts often contain content that should live elsewhere. The agent contract review (Section 9.5) evaluates both completeness and appropriate scope.

Appendix F - Glossary

Agent contract. The federation contract specialized for the Modus Tertius tier. The artifact that operationalizes the meta-harness for a specific persistent agent.

Audit lifecycle. The four-stage flow of audit information through the architecture: declaration in `morals.md` and `execution-policy.md`, emission in `execution-runtime.md`, storage and query in observability infrastructure, retention per `memory.md`.

Behavioral envelope. The bounded variation in outputs that a probabilistic substrate produces for a given input. The verification analog of output identity for deterministic systems.

Cognitive plane. The architectural plane responsible for agent orchestration, reasoning, planning, retrieval, memory, and evaluation. Fastest-changing layer in the architecture.

Federation contract. The structured specification of what a Modus tier inherits, delegates, escalates, and produces as conformance evidence. Format is consistent across tiers; content varies per tier.

Five-M decomposition. The meta-harness layer's organization into five files: `mind`, `morals`, `mission`, `memory`, `means`. The order is hierarchical: `mind` grounds `morals`, `morals` grounds `mission`, `mission` grounds `memory`, `memory` grounds `means`. Each file is a purposive declaration of one durable concern of the system's disposition.

Governance plane. The architectural plane responsible for identity, authorization, policy, approvals, audit, provenance, and compliance enforcement. Slowest-changing layer in the architecture.

Mechanism. Operationally agnostic machinery; available capacity independent of purpose. Distinct from means.

Means. Purposive disposition; what the system uses toward its mission within its morals. Distinct from mechanism.

Modus Primus through Quintus. The five tiers of the federation pattern: root governance, domain orchestration, federated specialists, tool-bound execution, ephemeral coordination.

Operational plane. The architectural plane responsible for tooling, infrastructure, enterprise systems, and runtime actions. Heterogeneous by nature.

PAHA. Portable Agent Harness Architecture (Longmire 2026; doi:10.5281/zenodo.20112631). The architectural framework this specification implements.

Runtime assurance. The architectural commitment to continuous monitoring of drift, mission coherence, policy deviation, and explainability. Serves as continuous V&V evidence (Section 7.4).

Selection relation. The relationship between purposive layers (meta-harness files) and mechanism layers (cognitive engine, orchestration, mechanism layer, execution-runtime). Purposive layers elect from mechanism layers.

Substrate substitution. A change to the cognitive substrate (foundation model replacement, routing policy revision) that triggers re-verification under Section 7.5.

Tech baseline. The per-enclave fulfillment record of the WBS specification. Specifies which products, services, or components fulfill each WBS entry.

Trust escalation. The progression from advisory to fully autonomous action through defined intermediate states. A governance plane primitive enforced uniformly across the federation.

WBS. Work Breakdown Structure; the technical baseline specification for what a Modus Primus instance comprises. Detailed in Appendix B; summarized in Section 3.

Document Closure

This is version 1.1 of the engineering specification companion to PAHA, incorporating adoption refinements over version 1.0. Refinements in v1.1 include an external standards mapping in Section 1.1, a PAHA-to-Modus mapping in Section 1.3, a First Actions by Audience subsection (Section 1.6), an Out of Scope subsection (Section 1.7), and a worked means election example in Section 6.3. The conventions adopted here are intended to support adoption across enterprise contexts with varying levels of existing engineering practice. Architectural commitments and engineering specializations stated here apply to AI architecture specifically.

Revisions to this specification are governed by the enterprise's configuration management process where one exists; in the absence of formal enterprise CM, revisions follow the revision discipline declared in the document's front matter. The next scheduled review is one year from the effective date.